

Universidad Nacional de Colombia

Facultad de Ingeniería



Ejercicios

Presentado por:

Leal Diaz Layonel Camilo

Jaekle Till

Profesor:

José J. Martínez P.

Asignatura:

Inteligencia Artificial y mini-robots

Bogotá, D.C

25 de marzo de 2026

Tarea 1 – Multiplicación de matrices con regresión lineal

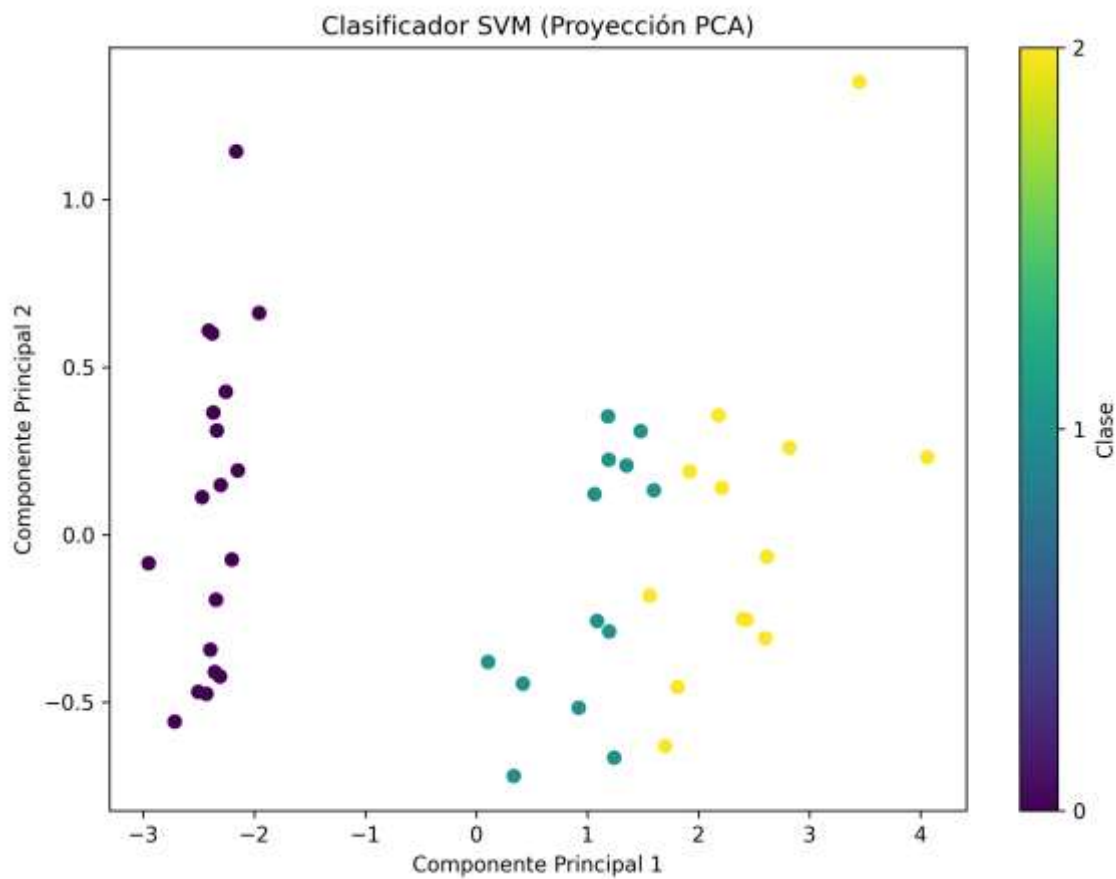
Se entrenó un modelo de regresión lineal para aproximar la multiplicación de dos matrices 2×2 con entradas enteras entre -20 y 20 . Para ello se generaron 50.000 ejemplos sintéticos, donde cada par de matrices A y B se aplanó en un vector de 8 características y el resultado $C = A \cdot B$ en un vector de 4 salidas. Se entrenaron cuatro modelos independientes, uno por cada componente de C , y se evaluaron sobre el 20% de datos reservado para prueba.

El MSE resultante fue de 39.280,76 y las diferencias elemento a elemento entre el resultado analítico y la predicción del modelo superaron en varios casos los 400 puntos, lo que demuestra que la regresión lineal no es capaz de aprender una operación no lineal como la multiplicación matricial. Como dato curioso, el modelo fue unas 100 veces más rápido en fase de inferencia que el cálculo analítico, aunque a costa de una pérdida enorme de precisión. En conclusión, ML no sustituye a algoritmos deterministas exactos cuando estos existen.

Tarea 2 – Clasificación con SVM

Se entrenó una SVM sobre el dataset Iris, usando GridSearchCV para optimizar automáticamente los hiperparámetros C , el tipo de kernel y gamma mediante validación cruzada. Los mejores parámetros encontrados fueron kernel = 'linear', $C = 1$ y gamma = 'scale', con los que el modelo alcanzó una exactitud del 100% sobre el conjunto de prueba, con precisión, recall y f1-score perfectos para las tres clases.

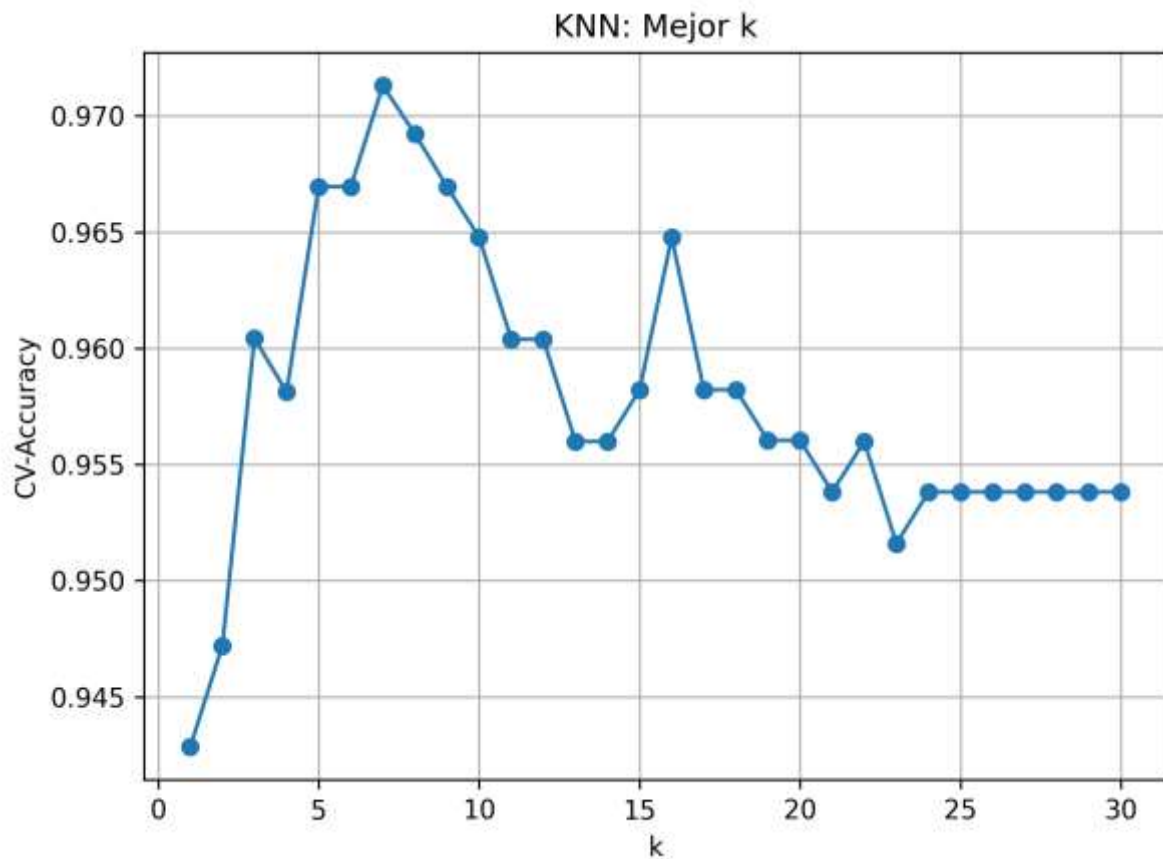
Para visualizar los resultados se aplicó PCA reduciendo las cuatro características originales a dos dimensiones y se proyectaron los puntos del conjunto de prueba coloreados según la clase predicha. La gráfica resultante se guardó en svm_iris_pca.png. Este ejercicio muestra que una SVM lineal bien ajustada puede resolver de forma perfecta un problema de clasificación relativamente sencillo como el del dataset Iris.



Tarea 3 – K-Nearest Neighbors (KNN)

Se utilizó el dataset Breast Cancer para clasificar tumores como benignos o malignos. Antes del entrenamiento se aplicó StandardScaler para normalizar las características, paso fundamental en KNN ya que el algoritmo se basa en distancias. Mediante validación cruzada de 10 folds se probaron valores de k entre 1 y 30, encontrando que $k = 7$ ofrece el mejor rendimiento con una CV-Accuracy de 0,971. Evaluado sobre el conjunto de prueba, el modelo definitivo alcanzó una exactitud de 0,947.

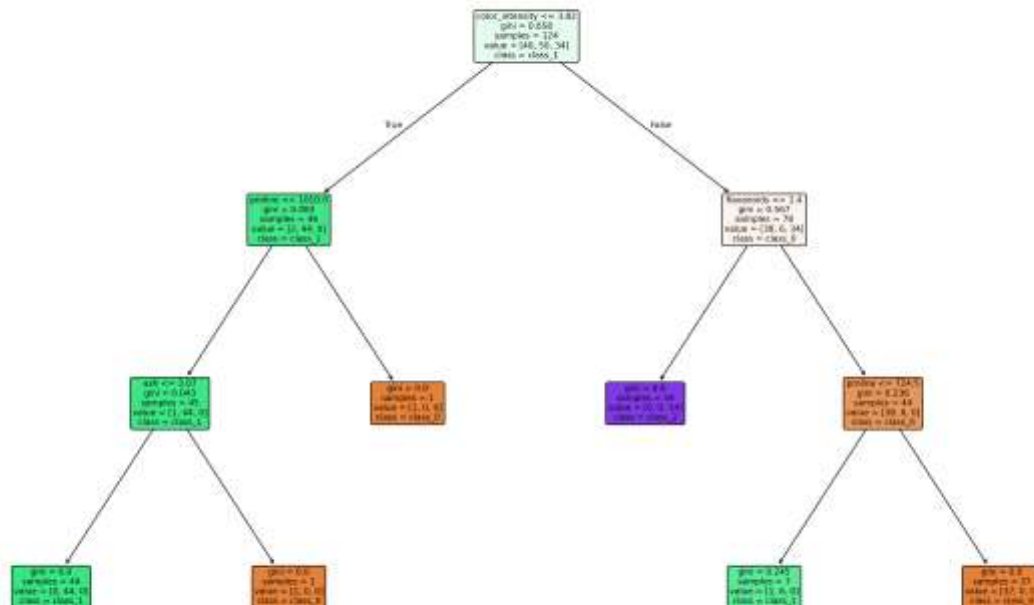
Se generó además una gráfica "codo" (accuracy vs. k) guardada en knn_elbow.png, que permite visualizar cómo el rendimiento varía con el número de vecinos. El ejercicio demuestra que KNN, con un buen preprocesamiento y una selección adecuada de k , es un clasificador sencillo pero muy eficaz.



Tarea 4 – Árboles de decisión

Se entrenó un DecisionTreeClassifier sobre el dataset Wine, que contiene 13 características químicas de tres tipos de vino. Para evitar el sobreajuste se aplicó una poda ligera fijando una profundidad máxima de 4 niveles y un mínimo de 10 muestras por nodo. El modelo alcanzó una exactitud del 96,3% en el conjunto de prueba, y el vector de importancias de características reveló que solo tres de las trece variables concentran prácticamente todo el peso en las decisiones del árbol.

El árbol entrenado se visualizó con plot_tree, con nodos coloreados por clase y umbrales de decisión visibles, y se guardó en decision_tree_wine.png. Este ejercicio resalta una ventaja clave de los árboles de decisión frente a otros algoritmos: además de ofrecer un alto rendimiento, producen un modelo completamente interpretable.



Tarea 5 – Naive Bayes

Se implementó un clasificador Naive Bayes multinomial sobre un pequeño conjunto sintético de diez mensajes de texto etiquetados como spam o ham. Los textos se transformaron en una representación de bolsa de palabras mediante CountVectorizer y se entrenó un modelo MultinomialNB. La exactitud obtenida sobre el conjunto de prueba fue de solo 0,5 (50%), con predicciones ['ham', 'spam', 'ham', 'ham'].